

MoodWave

개인화 음악 추천 스트리밍 플랫폼

박해준 | Frontend Developer

Spotify 로그인 → 음악 탐색 → 상황 기반 추천 → 재생·플레이리스트 관리

감정·날씨에 어울리는 곡과 최근 감상 이력에서 가장 많이 들은 장르의 인기곡을 추천합니다.

저는 공통 UI와 라우팅, Spotify OAuth·Web Playback SDK 플레이어, 검색·플레이리스트·감정 추천을 구현했습니다.

후속으로 요청 취소·캐싱·SDK 지연 로딩·이미지 최적화와 날씨·취향 AI 추천 API를 고도화했습니다.

1,339 ms → 173 ms

TBT 개선

51 → 97

Lighthouse Performance 개선

6.2 s → 2.0 s

LCP 개선

100

Accessibility / Best Practices

개인화 음악 추천 스트리밍 플랫폼

기간: 2026.05.26 - 2026.06.04 / 인원: 4명 / 역할: Frontend · Backend

프로젝트 소개

MoodWave는 사용자가 음악을 감상할 수 있는 스트리밍 웹 서비스로, 사용자의 감정과 현재 날씨에 어울리는 곡을 추천하고 사용자가 가장 많이 들은 장르의 인기곡을 제공하는 개인화 음악 추천 플랫폼입니다.

문제 정의

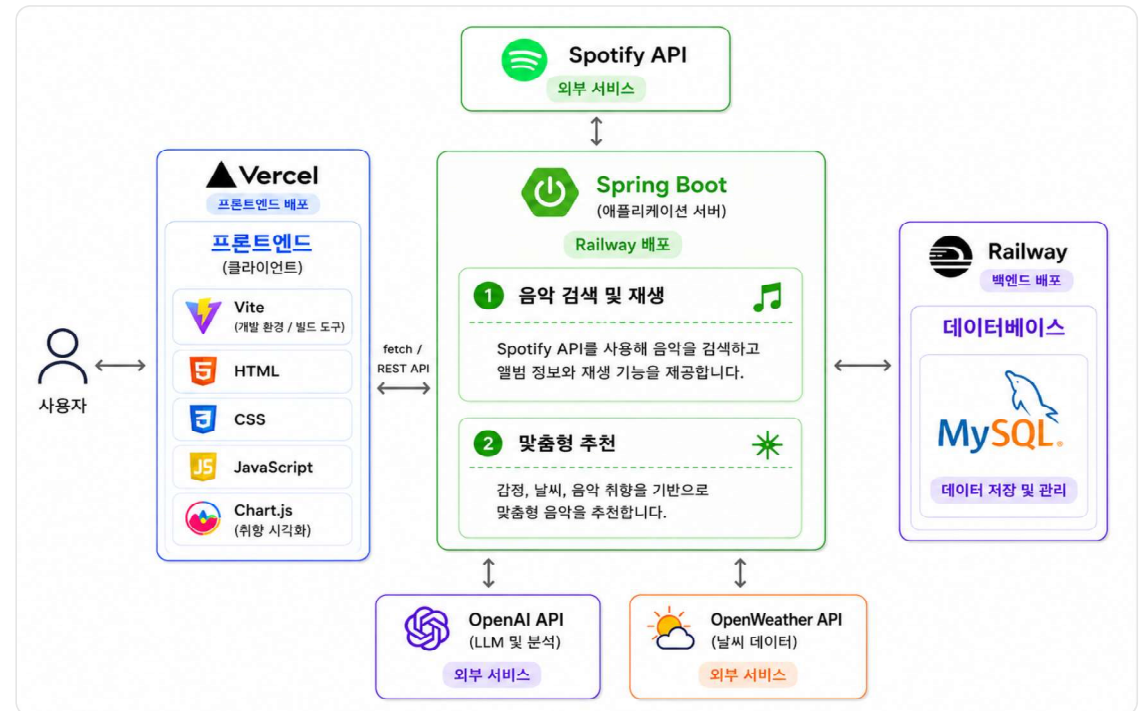
기존 음악 스트리밍 플랫폼은 인기 순위나 장르 중심의 추천에 그치는 경우가 많아, 사용자가 음악을 듣는 순간의 감정이나 날씨 같은 현재 상황을 반영하기 어렵다고 생각했습니다.

이를 보완하기 위해 MoodWave는 사용자의 감정과 현재 날씨에 어울리는 곡을 추천하고, 사용자가 가장 많이 들은 장르를 분석해 해당 장르의 인기곡을 함께 제공하는 개인화 음악 스트리밍 서비스로 구현했습니다.

사용한 기술 스택

Frontend: HTML, CSS, JavaScript, Vite, Chart.js
Backend: Java 17, Spring Boot, Security, MyBatis
API: Spotify Web API, OpenAI Responses API, OpenWeather
Deploy: Vercel, Railway, Railway MySQL

System Architecture



사용자 흐름과 담당 구현

음악을 찾는 순간부터 추천 곡을 재생하고 저장하는 과정까지 하나의 흐름으로 연결했습니다.



인증 / 라우팅

Spring Security OAuth2와 session cookie 기반 로그인을 연결했습니다.
hash 라우팅 전에 사용자 상태를 확인하고 보호 페이지 접근을 제어했습니다.

탐색 / 재생

홈·검색·Popular·Latest를 Spotify 데이터와 연결했습니다.
Web Playback SDK로 재생, 탐색, 음량, 셔플, 반복, 대기열을 구현했습니다.

추천 / 차트

사용자의 감정·현재 날씨에 어울리는 곡을 추천하고, 사용자가 가장 많이 들은 장르의 인기곡을 추천합니다.
사용자가 많이 듣는 장르와 날씨 플레이리스트 취향은 Chart.js로 시각화하였습니다.

핵심 구현

공통 레이아웃과 라우터, Spotify OAuth·Player, 검색, 좋아요, 플레이리스트, 감정 추천, 무한 스크롤 곡 테이블을 구현했습니다.

구조 개선

페이지 init 함수가 cleanup을 반환하도록 통일했습니다.
이벤트 해제, fetch abort, observer disconnect, Chart destroy를 라우트 생명주기에 연결했습니다.

기술 선택과 설계 판단

Vanilla JavaScript와 Vite

10일 일정과 학습 목표를 고려해 프레임워크 없이 UI와 상태 흐름을 직접 구현했습니다.
Vite로 ES Module, 환경변수, 로컬 proxy, production build를 구성했습니다.

페이지 init / cleanup

Main 영역만 교체하는 구조에서 재진입 시 작업이 누적되지 않도록
init 함수가 cleanup을 반환하게 했습니다.
이벤트·observer·요청·차트를 함께 정리합니다.

동일 출처 API 프록시

브라우저는 상대 경로만 사용하고 로컬은 Vite proxy, 배포는
Vercel rewrites가 Railway로 전달합니다.
OAuth와 session cookie 흐름을 통일했습니다.

공통 곡 테이블

Popular, Latest, Liked, AI 추천의 목록 동작을 통합했습니다.
IntersectionObserver로 다음 페이지를 미리 요청하고
TOP 버튼을 공통 제공했습니다.

인공지능 분석 / Spotify 검색

OpenAI는 사용자 맥락과 검색 키워드를 JSON Schema로 생성하고,
실제 재생 가능한 곡은 Spotify에서 조회합니다.
실패 시 규칙 기반 fallback을 사용합니다.

핵심 기능 구현

탐색, 재생, 추천, 저장을 같은 사용자 경험으로 연결했습니다.

Spotify 로그인과 플레이어

OAuth 로그인 후 Web Playback SDK로 재생·일시정지, 이전·다음 곡, 진행바, 음량, 셔플, 반복, 대기열, 기기 조화를 구현했습니다.
Spotify ID가 없으면 제목과 아티스트로 검색해 재생 URI를 찾습니다.

감정 기반 추천

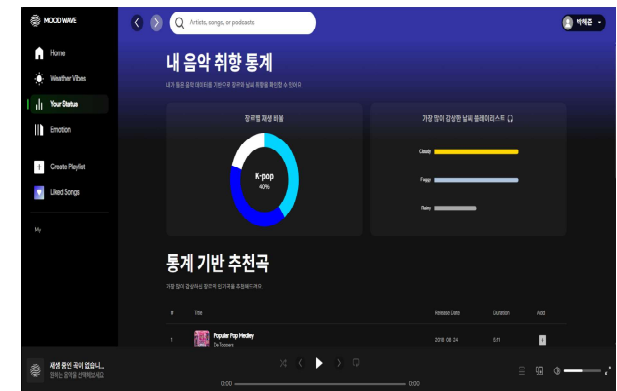
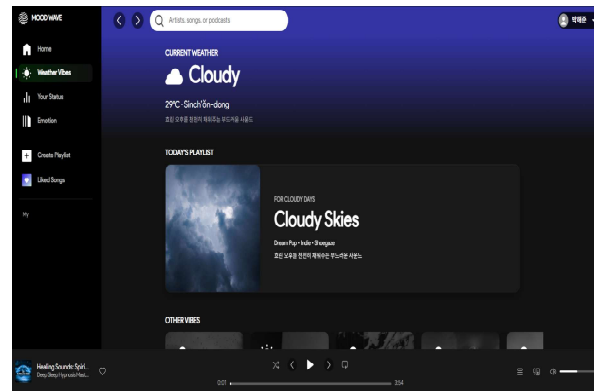
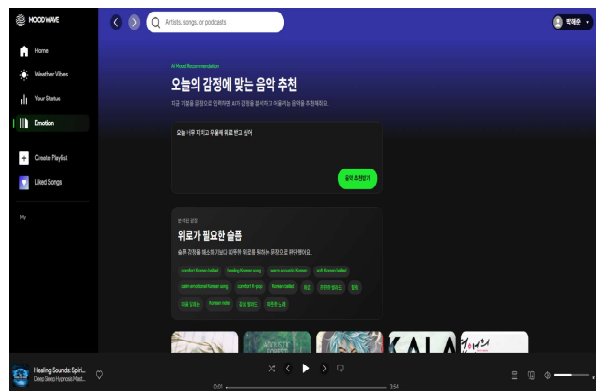
사용자의 감정 문장을 AI가 분석해 감정 라벨, 추천 이유, 키워드와 Spotify 추천 곡을 제공합니다.
재요청 시 이전 요청은 AbortController로 취소합니다.

날씨·취향 기반 추천

현재 날씨에 어울리는 곡을 추천하고, 사용자가 가장 많이 들은 장르의 인기곡을 최대 100곡 추천합니다.

플레이리스트와 차트

localStorage 기반 플레이리스트 생성·수정·삭제와 곡 관리를 구현했습니다.
사용자가 가장 많이 들은 장르와 날씨 플레이리스트 취향은 Chart.js로 시각화하고 페이지 진입 시에만 로드합니다.



Spotify Web Playback SDK Player

인증, 재생 제어, 상태 동기화, 검색 fallback을 하나의 Footer Player 흐름으로 연결했습니다.

인증과 SDK 준비

백엔드에서 Spotify access token을 받아 Web Playback SDK를 동적으로 로드합니다. token과 player 초기화 Promise를 공유해 중복 요청과 중복 player 생성을 방지했습니다.

재생 제어

재생·일시정지, 이전·다음, seek, 음량·음소거, 셔플·반복, 대기열, 기기 선택, 전체 화면을 Spotify API와 SDK에 연결했습니다.

Player state 동기화

player_state_changed 이벤트와 보정 타이머로 제목, 커버, 재생 시간, 진행바를 갱신했습니다. 401이면 token을 한 번 재발급해 요청을 재시도합니다.

트랙 검색 fallback

곡 데이터에 Spotify ID가 없으면 제목과 아티스트로 검색해 URI를 찾습니다. 검색 결과는 Map에 저장해 같은 곡의 반복 API 요청을 줄였습니다.

AI 추천 파이프라인

OpenAI + Spotify

AI는 사용자 맥락을 구조화하고 Spotify 검색 키워드를 생성합니다.
실제 재생 가능한 곡은 Spotify에서 조회하며 JSON Schema, 후처리, fallback으로 결과를 안정화했습니다.

감정 기반 추천

입력 문장을 감정 라벨, 추천 이유, 키워드로 분석합니다.
결과 텍스트는 escapeHTML 처리하고, 연속 요청에서는 이전 fetch를 취소합니다.

취향 기반 추천

Spotify 최근 재생 곡을 분석해 가장 많이 들은 대표 장르와 장르 분포를 생성합니다.
대표 장르의 인기곡 검색어로 최대 100곡을 수집합니다.

날씨 기반 추천

현재 날씨를 Sunny·Rainy·Foggy 등으로 정규화하고
날씨 분위기에 맞는 AI 검색어와 기본 프로필 검색어를 병합합니다.

100곡 추천 (취향 · 날씨 - 무한 스크롤)

키워드별 Spotify 검색 결과를 중복 제거하고 특정 아티스트를 최대 3곡으로 제한합니다.
최대 100곡을 page-limit 기준으로 분할 응답합니다.

Troubleshooting 1. 홈 LCP와 이미지 최적화

측정 → LCP 후보 확인 → 이미지 우선순위·크기 조정 → 배포 환경 재측정

문제

날씨 JPG 6개가 14.76 MiB였고, 홈 첫 화면 Popular 이미지에도 lazy loading이 적용돼 LCP 요청이 늦게 시작됐습니다.

해결

날씨 이미지를 WebP로 변환하고 원본 JPG를 제거했습니다.
첫 화면 이미지는 eager·fetchpriority=high, 하단 이미지는 lazy로 분리하고 Spotify 640px 커버를 300px로 변경했습니다.

14.76 Mib → 0.53 MiB

이미지 용량 개선

6.2 s → 2.0 s

LCP 개선

1,339 ms → 173 ms

TBT 개선

51 → 97

Lighthouse Performance 개선

모든 이미지에 lazy를 적용하지 않고 실제 LCP 후보와 화면 노출 순서를 기준으로 로딩 우선순위를 구분했습니다.

※ TBT와 Lighthouse Performance 개선에는 SDK 지연 로딩과 초기화 분산도 포함됩니다.

Troubleshooting 2. SDK 지연 로딩과 요청 캐싱

사용하지 않는 외부 리소스는 늦추고, 반복되는 데이터와 진행 중 요청은 공유했습니다.

외부 SDK 지연 로딩

Chart.js는 차트 페이지 진입 시, Spotify SDK는 실제 재생 시점에 script를 동적으로 추가했습니다.
홈 초기 네트워크 요청에서 두 SDK를 제거했습니다.

TTL cache

홈·차트·추천 5분, 날씨 좌표별 10분, 인증 사용자 30초 cache를 적용했습니다. token과 검색 결과도 메모리에서 재사용합니다.

Promise 공유

같은 시점의 SDK 초기화, access token, 홈·차트·트랙 검색 요청은 진행 중 Promise를 공유해 script 삽입과 API 요청 중복을 방지했습니다.

결과

페이지 재진입과 연속 재생에서 동일 요청을 줄이고, 외부 SDK 장애가 앱 초기 화면에 영향을 주는 범위를 축소했습니다.

Troubleshooting 3. 페이지 재진입 시 중복 작업

이벤트, observer, fetch, Chart instance를 페이지 생명주기 안에서 정리했습니다.

Before

이벤트 리스너 누적
Observer 중복 연결
이전 fetch 응답 렌더링

페이지를 다시 방문할수록 중복 실행과 오래된
UI 갱신 가능성 증가

After

`removeEventListener`
`AbortController + runId`
`observer.disconnect()` / `Chart.destroy()`

라우트 전환 전에 cleanup을 실행해 현재 화면의
작업만 유지

Troubleshooting 4. 곡 전환 상태 동기화

새 곡을 선택했을 때 이전 곡의 재생 위치가 잠시 노출되는 간헐적 오류를 해결했습니다.

Before

곡 정보: 새 곡
재생 위치: 이전 곡 재생 시간
진행바: 이전 값 유지

곡 정보와 Spotify player state의 갱신 시점이 달라
UI 상태가 불일치

After

재생 요청 직전 reset
시간 0:00 / 진행바 0%
SDK state 도착 후 실제 값 동기화

새 곡 화면에 이전 곡 시간이 노출되지 않고 실제
player state로 자연스럽게 전환

Troubleshooting 5. 취향 추천 장르 불일치

대표 장르가 K-pop일 때만 K-pop 인기곡 경로를 사용하도록 추천 규칙을 명확히 했습니다.

문제 원인

대표 장르가 Indie여도 AI 검색 키워드에 K-pop이 섞여 K-pop 인기곡이 추천될 수 있었습니다.
AI 응답을 그대로 검색에 사용한 것이 원인이었습니다.

추천 품질

특정 아티스트는 최대 3곡으로 제한하고, AI 실패 시 최근 재생 곡을 규칙 기반으로 분석합니다.
추천 결과는 5분 cache하고 최대 100곡을 분할합니다.

장르 분기

dominantGenre가 K-pop일 때만 K-pop 전용 검색어를 사용하고, 다른 장르에서는 K-pop 키워드를 제거해 해당 장르 기본 검색어를 추가했습니다.

검증 결과

Indie가 대표 장르일 때 K-pop 검색어와 K-pop 전용 검색 경로를 사용하지 않는 동작을 RecommendationServiceTest로 검증했습니다.

성능 및 품질 검증

Vercel 프론트와 Railway 백엔드가 연결된 실제 배포 주소에서 Lighthouse를 3회 측정했습니다.

97

Performance 중앙값

2.0 s

LCP 중앙값

173 ms

TBT 중앙값

100

Accessibility / Best Practices

측정 결과

Performance: 97 / 97 / 90, 중앙값 97
FCP 1.3초 · LCP 2.0초 · Speed Index 2.3초
TBT 173ms · CLS 0.003 · TTI 2.0초
/api/home, /api/user HTTP 200 · 콘솔 오류 0

빌드와 테스트

Vite production build 성공
JS 103.48 kB / gzip 27.40 kB
CSS 54.98 kB / gzip 10.09 kB

Playwright E2E 3개 통과 · 실패 0
보호 라우팅 · 감정 추천 · 오래된 비동기 응답 차단 검증

GitHub Actions CI · 배포 Smoke Test 통과
Vercel 프론트 · Railway 백엔드 /api/home 실연결 검증

Retrospective

비동기 작업의 시작과 종료 조건을 명확히 관리하는 방법을 배웠습니다.

배운 점

Spotify player state, API 응답, 페이지 이동, 진행 타이머는 서로 다른 시점에 동작합니다. cleanup, AbortController, runId, Promise 공유, TTL cache를 적용하며 화면 상태와 비동기 생명주기를 함께 설계했습니다.

개선점

footer.js가 인증, SDK, player state, UI, 이벤트를 함께 담당해 2,000줄 이상으로 커졌습니다. Player API, 상태 저장소, UI renderer로 분리하고 TypeScript를 적용하고 싶습니다. 또한, E2E 범위를 플레이어 상태 전환과 OAuth 실패 복구까지 확장하고 싶습니다.

MoodWave 프로젝트 한줄평

실제 음악 데이터를 연결하고,
비동기 생명주기를 정리하고,
측정 결과를 근거로 개선한 프로젝트

MoodWave

개인화 음악 추천 스트리밍 플랫폼

저는 이 프로젝트를 통해 실제 음악 데이터를 사용자 맥락과 연결하고,
비동기 작업의 생명주기를 정리하며, 측정 결과를 근거로 성능과 추천 품질을 개선했습니다.

Thank you

박해준 | Frontend Developer